Status	Finished
Started	Thursday, 1 October 2026, 1:58 PM
Completed	Thursday, 1 October 2026, 2:34 PM
Duration	35 mins 54 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```python
import sys

def valid_word_abbreviation(s: str, abbr: str) -> bool:
    i = 0  # Pointer for string s
    j = 0  # Pointer for abbr string

    m, n = len(s), len(abbr)

    while i < m and j < n:
        if abbr[j].isdigit():
            # Leading zero check: numbers like "0", "01", "02" are invalid
            if abbr[j] == '0':
                return False

            # Extract full number
            num = 0
            while j < n and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1

            # Skip 'num' characters in s
            i += num
        else:
            # Character comparison
            if s[i] != abbr[j]:
                return False
            i += 1
            j += 1

    # Both pointers must reach the exact ends of their respective strings
    return i == m and j == n

def main():
    # Read input lines, strip whitespace
    lines = [line.strip() for line in sys.stdin if line.strip()]
    if len(lines) >= 2:
        s = lines[0]
        abbr = lines[1]

        if valid_word_abbreviation(s, abbr):
            print("true")
        else:
            print("false")

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
import sys

def find_substring():
    input_data = sys.stdin.read().split()
    if len(input_data) >= 2:
        string1 = input_data[0]
        string2 = input_data[1]

        # find() returns the 0-based index of first occurrence, or -1 if not found
        result = string1.find(string2)
        print(result)

if __name__ == "__main__":
    find_substring()
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read input string from standard input
    s = sys.stdin.read().strip()
    if not s:
        return

    # Step 1: Separate USERNAME from the rest by splitting at '@'
    username, rest = s.split('@', 1)

    # Step 2: Separate DOMAIN from EXTENSION by splitting at the first '.' in 'rest'
    domain, extension = rest.split('.', 1)

    # Step 3: Print EXTENSION, DOMAIN, USERNAME in reverse order
    print(extension)
    print(domain)
    print(username)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
import sys

def is_palindrome(s: str) -> int:
    left, right = 0, len(s) - 1

    while left < right:
        # Move left pointer if character is not alphanumeric
        while left < right and not s[left].isalnum():
            left += 1

        # Move right pointer if character is not alphanumeric
        while left < right and not s[right].isalnum():
            right -= 1

        # Compare characters ignoring case
        if s[left].lower() != s[right].lower():
            return 0

        left += 1
        right -= 1

    return 1

def main():
    # Read entire input line including spaces
    s = sys.stdin.read().rstrip('\r\n')
    print(is_palindrome(s))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
import sys

def process_keystrokes(s: str) -> str:
    stack = []

    for char in s:
        if char == '^':
            if stack:
                stack.pop()
        else:
            stack.append(char)

    result = "".join(stack)
    return result if result else "-1"

def main():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # First line is N (number of test strings)
    n = int(input_data[0].strip())
```

```
23
24      for i in range(1, n + 1):
25          s = input_data[i]
26          print(process_keystrokes(s))
27
28  if __name__ == "__main__":
29      main()
30
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys

def main():
    keywords = {
        "break", "case", "continue", "default", "defer", "else",
        "for", "func", "goto", "if", "map", "range",
        "return", "struct", "type", "var"
    }

    # Read input string from stdin
    word = sys.stdin.read().strip()

    if not word:
        return

    # Check case-sensitive existence in keywords set
    if word in keywords:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys

def main():
    s = sys.stdin.read().strip()
    if not s:
        return

    result = []
    i = 0
    n = len(s)

    while i < n:
        if s[i].isalpha():
            char = s[i]
            i += 1

            # Extract full multi-digit count
            num = 0
            while i < n and s[i].isdigit():
                num = num * 10 + int(s[i])
                i += 1

            result.append(char * num)

    print("".join(result))

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> str:
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}

    for char in s:
        if char in mapping:
            top_element = stack.pop() if stack else '#'
            if mapping[char] != top_element:
                return "false"
        else:
            stack.append(char)

    return "true" if len(stack) == 0 else "false"
```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence: str) -> str:
    # Set of unique characters in the string
    unique_letters = set(sentence)

    # Check if all 26 lowercase English letters are present
    if len(unique_letters) == 26:
        return "true"
    else:
        return "false"
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓

	Test	Expected	Got	
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys

def reverse_string(s_list: list) -> list:
    left, right = 0, len(s_list) - 1
    while left < right:
        # Swap characters in-place
        s_list[left], s_list[right] = s_list[right], s_list[left]
        left += 1
        right -= 1
    return s_list

def main():
    input_str = sys.stdin.read().rstrip('\r\n')
    if not input_str:
        return

    # Convert string to array of characters
    char_array = list(input_str)

    # Reverse the array
    reverse_string(char_array)

    # Output the reversed string
    print("".join(char_array))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.